

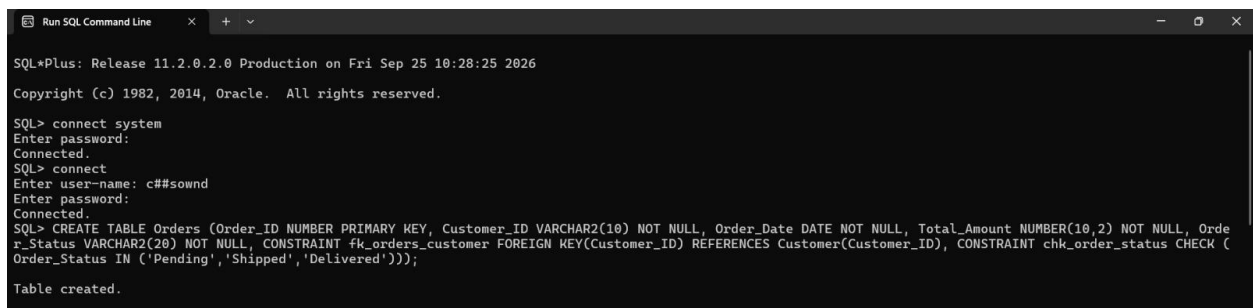
Week 4: Payment Management System

Scenario:

The Payment Management System manages payment details associated with customer orders. The system stores payment information such as payment date, payment mode, payment status, and transaction amount. It helps to track successful, failed, and pending transactions and allows failed payments to be updated after a retry.

1. Create Orders and Order_Details Tables

Orders:



```
Run SQL Command Line
SQL*Plus: Release 11.2.0.2.0 Production on Fri Sep 25 10:28:25 2026
Copyright (c) 1982, 2014, Oracle. All rights reserved.

SQL> connect system
Enter password:
Connected.
SQL> connect
Enter user-name: c##sownd
Enter password:
Connected.
SQL> CREATE TABLE Orders (Order_ID NUMBER PRIMARY KEY, Customer_ID VARCHAR2(10) NOT NULL, Order_Date DATE NOT NULL, Total_Amount NUMBER(10,2) NOT NULL, Order_Status VARCHAR2(20) NOT NULL, CONSTRAINT fk_orders_customer FOREIGN KEY(Customer_ID) REFERENCES Customer(Customer_ID), CONSTRAINT chk_order_status CHECK (Order_Status IN ('Pending','Shipped','Delivered')));
Table created.
```

Order_Details:

```
SQL> CREATE TABLE Order_Details (Order_Detail_ID NUMBER PRIMARY KEY, Order_ID NUMBER NOT NULL, Product_ID VARCHAR2(6) NOT NULL, Quantity NUMBER NOT NULL, Price NUMBER(10,2) NOT NULL, CONSTRAINT fk_od_order FOREIGN KEY(Order_ID) REFERENCES Orders(Order_ID), CONSTRAINT fk_od_product FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID), CONSTRAINT chk_od_quantity CHECK(Quantity > 0), CONSTRAINT chk_od_price CHECK(Price > 0));
```

Table created.

```
SQL> SELECT * FROM Customer;
```

CUSTOMER_ID	CUSTOMER_NAME
-------------	---------------

101	
-----	--

	PHONE
--	-------

Inserting values:

```
SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  3 VALUES
  4 (101, 'C001', TO_DATE('15-SEP-26','DD-MON-RR'), 499, 'Pending');
```

1 row created.

```
SQL>
```

```
SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  3 VALUES
  4 (102, 'C002', TO_DATE('16-SEP-26','DD-MON-RR'), 699, 'Shipped');
```

1 row created.

```
SQL>
```

```
SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  3 VALUES
  4 (103, 'C003', TO_DATE('17-SEP-26','DD-MON-RR'), 299, 'Delivered');
```

1 row created.

```
SQL>
```

```
SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  3 VALUES
  4 (104, 'C004', TO_DATE('18-SEP-26','DD-MON-RR'), 1198, 'Pending');
```

1 row created.

```
SQL>
```

```
SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  3 VALUES
  4 (105, 'C005', TO_DATE('19-SEP-26','DD-MON-RR'), 799, 'Shipped');
```

1 row created.

Order_Details:

```
SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3 VALUES
  4 (1, 101, 'P001', 1, 499);

1 row created.

SQL>
SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3 VALUES
  4 (2, 102, 'P002', 1, 699);

1 row created.

SQL>
SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3 VALUES
  4 (3, 103, 'P003', 1, 299);

1 row created.

SQL>
SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3 VALUES
  4 (4, 104, 'P004', 2, 599);

1 row created.

SQL>
SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3 VALUES
  4 (5, 105, 'P005', 1, 799);

1 row created.
```

3. Viewing Table structure:

```
SQL> SELECT * FROM Orders;
```

ORDER_ID	CUSTOMER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
101	C001	15-SEP-26	499	Pending
102	C002	16-SEP-26	699	Shipped
103	C003	17-SEP-26	299	Delivered
104	C004	18-SEP-26	1198	Pending
105	C005	19-SEP-26	799	Shipped

```
SQL> SELECT * FROM Order_Details;
```

ORDER_DETAIL_ID	ORDER_ID	PRODUC	QUANTITY	PRICE
1	101	P001	1	499
2	102	P002	1	699
3	103	P003	1	299
4	104	P004	2	599
5	105	P005	1	799

```
SQL> commit complete
```

```
2
```

```
SQL> commit;
```

```
Commit complete.
```

4. Viewing Values in Table: Orders:

```
SQL> SELECT * FROM Orders;
```

ORDER_ID	CUSTOMER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
101	C001	15-SEP-26	499	Pending
102	C002	16-SEP-26	699	Shipped
103	C003	17-SEP-26	299	Delivered
104	C004	18-SEP-26	1198	Pending
105	C005	19-SEP-26	799	Shipped
107	C002	21-SEP-26	599	Pending
108	C003	22-SEP-26	1299	Shipped
109	C004	23-SEP-26	749	Delivered
110	C005	24-SEP-26	999	Pending
111	C001	25-SEP-26	449	Shipped

```
10 rows selected.
```

Order_Details:

```
SQL> SELECT * FROM Order_Details;
```

ORDER_DETAIL_ID	ORDER_ID	PRODUC	QUANTITY	PRICE
1	101	P001	1	499
2	102	P002	1	699
3	103	P003	1	299
4	104	P004	2	599
5	105	P005	1	799
7	107	P002	1	599
8	108	P003	2	299
9	109	P004	1	749
10	110	P005	2	999
11	111	P001	1	449

```
10 rows selected.
```

1. Create Payment Table

The Payment table is created to store payment transaction details.

Attributes:

- Payment_ID
- Order_ID
- Payment_Date
- Payment_Mode
- Payment_Status
- Transaction_Amount

2. Establish Relationship

Orders and Payment

- One order can have a payment transaction.
- Payment is associated with an existing order.
- Order_ID in the Payment table is used as a Foreign Key referencing the Orders table.

Relationship:

Orders → Payment

3. Constraints

Primary Key

Payment_ID is defined as the Primary Key to uniquely identify each payment.

Foreign Key

Order_ID is defined as a Foreign Key referencing Orders(Order_ID).

NOT NULL Constraints

The following attributes are made NOT NULL:

- Payment_ID
- Order_ID
- Payment_Date
- Payment_Mode
- Payment_Status
- Transaction_Amount

Payment Mode Constraint

The Payment Mode is restricted to:

- UPI
- Card
- Net Banking
- Cash

Payment Status Constraint

The Payment Status is restricted to:

- Success
- Failed
- Pending

Transaction Amount Constraint

Transaction amount must be greater than zero.

4. Insert Values into Payment Table

Payment records are inserted with different payment modes and statuses such as **UPI, Card, Cash, Net Banking, Success, Failed, and Pending**.

The Payment table contains payment details corresponding to existing Order IDs.

5. Update Payment Details

A failed payment can be updated after a successful retry.

For example, the status of Payment ID 11 is changed from **Failed** to **Success** after retry.

6. Delete Payment Record

A payment record can be deleted using its unique `Payment_ID`.

```
SQL> UPDATE Order_Details
  2  SET Price = 950
  3  WHERE Order_Detail_ID = 6;

0 rows updated.

SQL> UPDATE Order_Details
  2  SET Price = 950
  3  WHERE Order_Detail_ID = 5;

1 row updated.

SQL> UPDATE Order_Details
  2  SET Quantity = 3
  3  WHERE Order_Detail_ID = 7;

1 row updated.

SQL> COMMIT;

Commit complete.
```

7.Report 1: Customer Order History Displays:

- Customer Name
- Order ID
- Order Date
- Total Amount
- Order Status QUERY

```
: SELECT Customer.Customer_Name, Orders.Order_ID, Orders.Order_Date,
Orders.Total_Amount, Orders.Order_Status
FROM Customer
JOIN Orders
```

ON Customer.Customer_ID = Orders.Customer_ID;

-This report combines customer and order information to display the complete order history of customers.

```
SQL> SELECT c.Customer_ID,  
2         c.Customer_Name,  
3         COUNT(o.Order_ID) AS Total_Orders  
4 FROM Customer c  
5 JOIN Orders o  
6 ON c.Customer_ID = o.Customer_ID  
7 GROUP BY c.Customer_ID, c.Customer_Name  
8 ORDER BY Total_Orders DESC;
```

CUSTOMER_I	CUSTOMER_NAME	TOTAL_ORDERS
C002	Ananya	2
C001	Rahul	2
C004	Meena	2
C005	Karthik	2
C003	Vikram	2

SQL>

9. Report 2: Product-wise Order Report Finds:

Finds:

Number of times each product was ordered.

Total quantity of each product sold.

QUERY: SELECT Product.Product_ID, Product.Product_Name,

COUNT(Order_Details.Order_ID) AS Times_Ordered,

SUM(Order_Details.Quantity) AS Total_Quantity_Sold

FROM Product

JOIN Order_Details

ON Product.Product_ID = Order_Details.Product_ID

GROUP BY Product.Product_ID, Product.Product_Name;

This report helps identify how frequently each product is ordered and the total number of units sold.

Report 3: Customer Purchase Analysis

❖ Customers with Maximum Orders:

QUERY: SELECT Customer.Customer_ID, Customer.Customer_Name,
 COUNT(Orders.Order_ID) AS Total_Orders
 FROM Customer
 JOIN Orders
 ON Customer.Customer_ID = Orders.Customer_ID
 GROUP BY Customer.Customer_ID, Customer.Customer_Name
 ORDER BY Total_Orders DESC;

```
SQL> SELECT c.Customer_ID,
2         c.Customer_Name,
3         SUM(o.Total_Amount) AS Total_Spending
4 FROM Customer c
5 JOIN Orders o
6 ON c.Customer_ID = o.Customer_ID
7 GROUP BY c.Customer_ID, c.Customer_Name
8 ORDER BY Total_Spending DESC;
```

CUSTOMER_I	CUSTOMER_NAME	TOTAL_SPENDING
C004	Meena	1947
C005	Karthik	1798
C003	Vikram	1598
C002	Ananya	1298
C001	Rahul	948

Report 2: Product-wise Order Report Finds: Number of times each product was ordered

. Total quantity of each product sold.

QUERY: SELECT Product.Product_ID, Product.Product_Name,
 COUNT(Order_Details.Order_ID) AS Times_Ordered,
 SUM(Order_Details.Quantity) AS Total_Quantity_Sold
 FROM Product JOIN Order_Details ON Product.
 Product_ID = Order_Details.Product_ID
 GROUP BY Product.Product_ID, Product.Product_Name;

This report helps identify how frequently each product is ordered and the total number of units sold.

. Report 3: Customer Purchase Analysis ❖ Customers with Maximum Orders:

QUERY: SELECT Customer.Customer_ID Customer.Customer_Name,
 COUNT(Orders.Order_ID) AS Total_Orders

FROM Customer

JOIN Orders

ON Customer.Customer_ID = Orders.Customer_ID

GROUP BY Customer.Customer_ID, Customer.Customer_Name ORDER BY Total_Orders
DESC;

```
SQL> SELECT c.Customer_ID,  
2         c.Customer_Name,  
3         COUNT(o.Order_ID) AS Total_Orders  
4 FROM Customer c  
5 JOIN Orders o  
6 ON c.Customer_ID = o.Customer_ID  
7 GROUP BY c.Customer_ID, c.Customer_Name  
8 ORDER BY Total_Orders DESC;
```

CUSTOMER_I	CUSTOMER_NAME	TOTAL_ORDERS
C002	Ananya	2
C001	Rahul	2
C004	Meena	2
C005	Karthik	2
C003	Vikram	2

SQL >

This report calculates the number of orders placed by each customer.

❖ Total Spending by Each Customer:

QUERY: SELECT Customer.Customer_ID, Customer.Customer_Name,

SUM(Orders.Total_Amount) AS Total_Spending

FROM Customer

JOIN Orders

ON Customer.Customer_ID = Orders.Customer_ID

GROUP BY Customer.Customer_ID, Customer.Customer_Name;

```

SQL> SELECT c.Customer_ID,
2         c.Customer_Name,
3         SUM(o.Total_Amount) AS Total_Spending
4 FROM Customer c
5 JOIN Orders o
6 ON c.Customer_ID = o.Customer_ID
7 GROUP BY c.Customer_ID, c.Customer_Name
8 ORDER BY Total_Spending DESC;

```

CUSTOMER_I	CUSTOMER_NAME	TOTAL_SPENDING
C004	Meena	1947
C005	Karthik	1798
C003	Vikram	1598
C002	Ananya	1298
C001	Rahul	948

-This report calculates the total amount spent by each customer

```

SQL> SELECT c.Customer_ID,
2         c.Customer_Name,
3         AVG(o.Total_Amount) AS Average_Order_Value
4 FROM Customer c
5 JOIN Orders o
6 ON c.Customer_ID = o.Customer_ID
7 GROUP BY c.Customer_ID, c.Customer_Name
8 ORDER BY Average_Order_Value DESC;

```

CUSTOMER_I	CUSTOMER_NAME
------------	---------------

AVERAGE_ORDER_VALUE

C004	Meena	973.5
------	-------	-------

C005	Karthik	899
------	---------	-----

C003	Vikram	799
------	--------	-----

CUSTOMER_I	CUSTOMER_NAME
------------	---------------

AVERAGE_ORDER_VALUE

C002	Ananya	649
------	--------	-----

C001	Rahul	474
------	-------	-----

-This report calculates the average value of orders placed by each customer.